

CI Pipeline Architecture

Docker-based Build, Test & Deployment Flow

Stage	Action	Technology	Output
1	Code pushed to repository	Git (feature branch → main)	Trigger event fired
2	CI pipeline triggered	Webhook / CI trigger (e.g. GitHub Actions, Jenkins)	Pipeline job started
3	Application built into image	Docker build	Docker image created
4	Automated tests run inside container	Automated test suite (unit + integration)	Pass/fail test report
5	Manual testing checkpoint (pre-release)	Manual QA on staging build	Exploratory / UX validation
6	Image pushed to registry	Docker push (if tests pass)	Versioned image stored
7	Deployment to environment	Docker run / orchestrator pull	App live in target environment
8	Status reported back	CI dashboard / notification	Build success/failure visible to team

How This Fits Together

The CI pipeline starts when code is pushed — this automatically triggers a build where Docker packages the application into an image containing its exact runtime environment. Inside the same pipeline, automated tests run against the container to catch regressions immediately. If those pass, a manual testing checkpoint is performed on staging to catch usability and edge-case issues that automation cannot script. Once both are clear, the image is pushed to a registry and deployed. If any stage fails, the pipeline stops and reports back immediately, so issues are caught before reaching production.